

GN 爐粒子噴射互動網站 開發計畫書

GN DRIVE PARTICLE SIMULATION & INTERACTIVE WEB DESIGN MASTER BLUEPRINT

專案目標：打造《鋼彈00》3D GN Drive WebGL 體驗與教學網站

核心技術：Three.js + GLSL Shaders + GSAP + GitHub Pages

執行方式：GitHub Codespace + AI Agent 循序漸進自動化開發

部署目標：GitHub Pages 自動化 CI/CD

1. 專案概述與架構規劃

本專案旨在運用現代 WebGL 最佳實踐，打造一個具備高度視覺衝擊力與流暢互動感的《機動戰士鋼彈00》GN 爐（GN Drive）粒子噴射網站。內容不僅包含完整的 3D 特效體驗，亦封裝成分步教學模組，方便使用者探索 3D 視覺與數學模型的運算機制。

模組名稱	技術細節 / 實現方式	視覺/互動目標
3D 爐體模型	Three.js GLTFLoader / MeshPrimitive	金屬錐型外殼、內部發光核心、質感貼圖
GPU 粒子系統	THREE.Points + Custom GLSL Shaders	數萬粒子的高效能軸向與螺旋渦流噴射
發光後處理	EffectComposer + UnrealBloomPass	真實的高能量 Glow/Bloom 綠螢光與紅光爆發
狀態控制 (GSAP)	GSAP Timeline & Uniform Interpolation	一般模式與 Trans-AM 模式的平滑過渡

2. 物理與數學模型 (Mathematical Model)

為達成鋼彈00中 GN 粒子靈動且具渦流感的噴射視覺，GPU Vertex Shader 將依據以下四重數學模型即時計算每個粒子的空間座標：

$$P(t) = P_0 + V_{axial}(t) + R_{vortex}(t) + D_{noise}(P, t)$$

2.1 四大核心運算公式：

• 1. 軸向錐形推進 (Axial & Conical Expansion)：

沿著主軸 z 推進： $V_z = v_0 \cdot t + \frac{1}{2} a \cdot t^2$ 。同時搭配錐形擴散因子 $x_{cone} = r_0 \cdot \cos(\varphi) \cdot (1 + k \cdot z)$ 。

• 2. 螺旋渦流場 (Vortex Angular Velocity)：

角速度隨距離遞減： $\omega(z) = \omega_0 \cdot e^{(-\lambda z)}$ 。粒子剛噴出時急速旋轉，向外散開時轉速逐漸放緩。

• 3. 3D Simplex 雜訊力場 (Turbulence / Noise)：

$D_{noise} = A \cdot Simplex3D(P / \lambda_{noise} + f \cdot t)$ ，賦予粒子流體般自然的亂流飄散感。

• 4. 非線性 Alpha 與尺寸衰減：

$\alpha(T) = 4T(1 - T) \cdot e^{(-\gamma T)}$ ，確保粒子自中心生成時漸亮、散開時自然消逝。

3. Codespaces 指令集與 Agent 執行 workflow (Prompt Guide)

當你在 GitHub Codespace 中打開專案時，只需將本 PDF 放置於根目錄，並複製以下標準 Prompt 逐步告訴 Agent 「繼續」即可完成開發：

💡 啟動對話指令 (First Prompt to Agent)

「請讀取專案根目錄下的《GN 爐粒子噴射互動網站 開發計畫書 PDF》，按照內容規劃，一步一步開始撰寫程式碼並建立網頁專案。當我輸入『繼續』時，請自動執行下一個階段的開發 Task。」

階段式 Task 拆解 (Agent 開發步驟)：

1. **Task 1** 初始化專案結構：建立 `index.html`, `style.css`, `src/main.js`, `src/shaders/` 並設定 Vite 構建工具。
2. **Task 2** 搭建 Three.js 基礎場景：設置 Camera, Scene, WebGLRenderer 與 OrbitControls，創建 3D GN Drive 爐體基本幾何形狀。
3. **Task 3** 撰寫 GLSL Particle Shader：建立 Vertex & Fragment Shader，實現數學模型的軸向、渦流與 Noise 運算。
4. **Task 4** 整合 UnrealBloomPass：設置 EffectComposer 後處理，打造高質感的發光輝光效果。
5. **Task 5** 實現 Trans-AM 切換：使用 GSAP 綁定按鈕，達成綠光與紅光（3倍速/高強度）平滑切換動畫。
6. **Task 6** UI 介面與教學面板：加入控制面板與分頁說明，讓網頁具備教學與展示雙重價值。
7. **Task 7** GitHub Actions 部署：撰寫 `.github/workflows/deploy.yml` 實現自動部署至 GitHub Pages。

4. GitHub Pages 自動化部署 Workflow

建立 `.github/workflows/deploy.yml`，每次推送主分支（push to main）即自動建置並部署：

```
name: Deploy GN Drive Web to GitHub Pages

on:
  push:
    branches: [ main ]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: 18

      - name: Install Dependencies & Build
        run: |
          npm install
          npm run build

      - name: Deploy to GitHub Pages
        uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${ secrets.GITHUB_TOKEN }
          publish_dir: ./dist
```

5. 驗收與最佳實踐檢核清單 (QA Checklist)

- [] 效能檢測：畫面於桌面端需穩定維持 60 FPS（粒子數 30,000+）。
- [] 響應式適應：行動端自動降級粒子數量至 10,000，確保流暢度。
- [] Trans-AM 切換：顏色、速度、Bloom 強度皆具備平滑 GSAP 漸變，無突兀跳幀。
- [] 記憶體管理：切換或重置場景時完整釋放 Geometry 與 Material，無 WebGL 洩漏。